COSC364-25S1
Internet Technology and Engineering
RIPv2 Assignment

Apr 14, 2025

Shunzhi Zhang（31340795）
Yumeng Shi（74341182）

Table of Contents

Declaration

I declare that this assignment submission represents my own work (except for allowed material provided in the course), and that ideas or extracts from other sources are properly acknowledged in the report. I have not allowed anyone to copy my work with the intention of passing it off as their own work.

Name: Yumeng Shi
Student ID: 74341182
Date: 16/4/2025

Name: Shunzhi Zhang
Student ID: 31340795
Date: 16/4/2025

1 Summary

This programming project is a routing daemon developed in Python 3, utilizing a socket interface on the Linux operating system to implement Routing Information Protocol Version 2 (RIPv2). It includes modules for configuration processing, packet handling, routing algorithms, and server functionality. The project features a user-friendly interface and is well-structured to facilitate the addition of new functions and future maintenance.

1.1 Reflection

The project is well-structured, featuring a modular design with distinct functions and modules such as routing_table.py, routing_algorithms.py, and config_processer.py. This modular approach is effectively applied in the Server function, which runs the overall functionality logic. The separation of concerns into different modules makes the code easier to read, maintain, and debug. Each module handles a specific aspect of the router's operation, contributing to a clear and organized architecture. This design choice not only enhances the maintainability of the code but also facilitates easier debugging and potential future enhancements.

For functional implementation, we use the same structure as the routing table to initialize the original routing information, ensuring configuration data is stored efficiently and avoiding duplicate file reads. We also have a neighbor mapping utility to match neighbor ports and IDs, as these two variables are frequently used.

For time management, we utilize Python's time interface to generate the current time and the next second. We employ Python's random module to create a next_periodic_update_time with a variance of 30 seconds ± 5 seconds. When current_time is greater than or equal to next_second, it indicates that one second has passed. Similarly, when current_time is greater than or equal to next_periodic_update_time, it's time to trigger an update.

We implement split-horizon with poisoned reverse and create different poisoned and non-poisoned packets for the router’s neighbors. For updates, we have periodic updates every 30 seconds and triggered updates when there is a cut-off (bad news) in the path. No "good news" is sent immediately after a routing table update. This achieves the RIPv2 behavior: "bad news travels fast but good news travels slow."

1.2 Contributed

For the entire project, we both agreed that each of us contributed equally, with a 50% share each.

Yumeng Shi:
Planning
Testing – Unit Tests
Coding – Main Server Function, Routing Table, and Algorithm
Report writing

Shunzhi Zhang:
Planning
Coding – Configuration Processing and RIP Packet
Testing – Integration Tests
Configuration File Development

2 Testing

2.1 Unit Test

Testing individual components of the code in isolation to ensure they behave as expected.

Config Processer

We test all the router configuration files used for integration tests with the big model(should not raise any error). We create bad configuration files (have error raise) with invalid input ports, invalid router IDs, invalid output ports, and empty files.

```
#test()

##########################test cases#########################

def test_router1():

    config_filename = 'router1.txt'

    config = read_config(config_filename)

    assert config == {'router_id': 1, 'input_ports': [1111, 1112, 1113], 'output_ports':

[[2221, 1, 2], [6661, 5, 6], [7771, 8, 7]]}

    print("router1 pass.")


def test_router2():

    config_filename = 'router2.txt'

    config = read_config(config_filename)

    assert config == {'router_id': 2, 'input_ports': [2221, 2222], 'output_ports': [[1111, 1,

1], [3331, 3, 3]]}

    print("router3 pass.")
```

```
def test_router3():
    config_filename = 'router3.txt'
    config = read_config(config_filename)
    assert config == {'router_id': 3, 'input_ports': [3331, 3332], 'output_ports': [[2222, 3,
2], [4441, 4, 4]]}
    print("router3 pass.")

def test_router4():
    config_filename = 'router4.txt'
    config = read_config(config_filename)
    assert config == {'router_id': 4, 'input_ports': [4441, 4442, 4443], 'output_ports':
[[3332, 4, 3], [7772, 6, 7], [5551, 2, 5]]}
    print("router4 pass.")

def test_router5():
    config_filename = 'router5.txt'
    config = read_config(config_filename)
    assert config == {'router_id': 5, 'input_ports': [5551, 5552], 'output_ports': [[4442, 2,
4], [6662, 1, 6]]}
    print("router5 pass.")

def test_router6():
    config_filename = 'router6.txt'
    config = read_config(config_filename)
    assert config == {'router_id': 6, 'input_ports': [6661, 6662], 'output_ports': [[1112, 5,
1], [5552, 1, 5]]}
    print("router6 pass.")

def test_router7():
    config_filename = 'router7.txt'
    config = read_config(config_filename)
    assert config == {'router_id': 7, 'input_ports': [7771, 7772], 'output_ports': [[1113, 8,
1], [4443, 6, 4]]}
```

```
    print("router7 pass.")


def test_invalid_input_port():
    config_filename = 'invalid_input_port.txt'
    try:
        config = read_config(config_filename)
    except ValueError as e:
        expected_message = "ERROR: Invalid input_ports format, input_ports must be less than
64000, not 1111111"
        assert str(e) == expected_message
        print("Correct error raised:", e)


def test_invalid_router_id():
    config_filename = 'invalid_router_id.txt'
    try:
        config = read_config(config_filename)
    except ValueError as e:
        expected_message = "ERROR: Invalid router_id format, router_id must be not an
alphabetic string, not abc"
        assert str(e) == expected_message
        print("Correct error raised:", e)


def test_invalid_output_port():
    config_filename = 'invalid_output_port.txt'
    try:
        config = read_config(config_filename)
    except ValueError as e:
        expected_message = "ERROR: Invalid output-port format, must be 'output-ports
<peer_port> <metric> <peer_ID>'"
        assert str(e) == expected_message
        print("Correct error raised:", e)


def test_empty():
    config_filename = 'empty.txt'
```

```
    try:
        config = read_config(config_filename)
    except ValueError as e:
        expected_message = "ERROR: Invalid router config file, empty file"
        assert str(e) == expected_message, f"Unexpected error message: {e}"
        print("Correct error raised:", e)
```

---

invalid_input_port.txt

```
router-id 1
input-ports 1111111, 1112, 1113
output-ports 2221-1-2-3, 6661-5-6, 7771-8-7
def test_invalid_input_port():
    config_filename = 'invalid_input_port.txt'
    try:
        config = read_config(config_filename)
    except ValueError as e:
        expected_message = "ERROR: Invalid input port 1111111. Must be in range 1024-64000."
        assert str(e) == expected_message, f"Unexpected error message: {e}"
        print("Correct error raised:", e)
```

---

invalid_router_id.txt

```
router-id abc
input-ports 1111, 1112, 1113
output-ports 2221-1-2, 6661-5-6, 7771-8-7
def test_invalid_router_id():
    config_filename = 'invalid_router_id.txt'
    try:
        config = read_config(config_filename)
    except ValueError as e:
        expected_message = "ERROR: router_id must be an integer."
        assert str(e) == expected_message, f"Unexpected error message: {e}"
        print("Correct error raised:", e)
```

---

invalid_output_port.txt

```
router-id 1
```

input-ports 1111, 1112, 1113

output-ports 2221-1-2-3, 6661-5-6, 7771-8-7

```
def test_invalid_output_port():
    config_filename = 'invalid_output_port.txt'
    try:
        config = read_config(config_filename)
    except ValueError as e:
        expected_message = "ERROR: Invalid output-port format"
        assert str(e) == expected_message, f"Unexpected error message: {e}"
        print("Correct error raised:", e)
```

---

empty.txt

```
def test_empty():
    config_filename = 'empty.txt'
    try:
        config = read_config(config_filename)
    except ValueError as e:
        expected_message = "ERROR: Configuration file is empty."
        assert str(e) == expected_message, f"Unexpected error message: {e}"
        print("Correct error raised:", e)
```

```
jadeshi@jades-mbp assignment1 % /opt/homebrew/bin/python3 /Users/jadeshi/Desktop/COSC364/assignment1/test/test_config_processer.py
router1 pass.
router3 pass.
router3 pass.
router4 pass.
router5 pass.
router6 pass.
router7 pass.
Correct error raised: ERROR: Invalid input_ports format, input_ports must be less than 64000, not 1111111
Correct error raised: ERROR: Invalid router_id format, router_id must be not an alphabetic string, not abc
Correct error raised: ERROR: Invalid output-port format, must be 'output-ports <peer_port> <metric> <peer_ID>'
Correct error raised: ERROR: Invalid router config file, empty file
```

RIP packet

Test functionality of different components related to RIP, include header, entry ,full packet and poisoned packet been build.

```
def test_RIP_header():
    router_id = 1
    exp_header = [COMMAND_REQUEST, VERSION, router_id]
    header = RIP_header(router_id)
    assert header == exp_header
```

```
    print("test_RIP_header pass.")
def test_RIP_entry():
    table = {
        3: {'next_hop': 5, 'cost': 2, 'garbage': False},
        4: {'next_hop': 4, 'cost': 3, 'garbage': False},
        5: {'next_hop': 5, 'cost': 4, 'garbage': True}  # This entry should be ignored
    }
    exp_entries = [[3, 2], [4, 3]]
    entries = RIP_entry(table)
    assert entries == exp_entries
    print("test_RIP_entry pass.")
def test_rip_packet():
    router_id = 1
    table = {
        3: {'next_hop': 5, 'cost': 2, 'garbage': False},
        4: {'next_hop': 4, 'cost': 3, 'garbage': False}}
    exp_packet = {
        'header': [COMMAND_REQUEST, VERSION, router_id],
        'entry': [[3, 2], [4, 3]] }
    packet = rip_packet(router_id, table)
    assert packet == exp_packet
    print("test_rip_packet pass.")
def test_set_poisoned_reverse():
    #Send to router 5, with a router 3 use next hop 5, should set to 16
    router_ID = 1
    table = {
        3: {'next_hop': 5, 'cost': 2, 'garbage': False},
        4: {'next_hop': 4, 'cost': 3, 'garbage': False},
        5: {'next_hop': 5, 'cost': 4, 'garbage': False}}
    port_id = 5
    exp_packet = {
        'header': [COMMAND_REQUEST, VERSION, router_ID],
        'entry': [(3, 16), (4, 3), (5, 4)]}
    poisoned_packet = set_poisoned_reverse(router_ID, table, port_id)
```

```
    assert poisoned_packet == exp_packet

    print("test_set_poisoned_reverse pass.")
```

```
jadeshi@jades-mbp assignment1 % /opt/homebrew/bin/python3 /Users/jadeshi/Desktop/COSC364/assignment1/test/test_RIP_packet.py
test_RIP_header pass.
test_RIP_entry pass.
test_rip_packet pass.
test_set_poisoned_reverse pass.
```

RIP packet Receiver

For check the consistency in Server when RIP packet received, we test the invalid command(not 2), version(not 2), header length, destination id, metric
And empty entry.

```
import Server as s

def test_invalid_command():

    pkt = {

        'header': [1, 2, 12345],

        'entry': [[54321, 5]]

    }

    try:

        s.check_pkt(pkt)

        print("Invalid command  test , No error raised!!")

    except ValueError:

        print("Invalid command test pass")


def test_invalid_version():

    pkt = {

        'header': [2, 3, 12345],

        'entry': [[54321, 5]]

    }

    try:

        s.check_pkt(pkt)

        print("Invalid version test , No error raised!!")

    except ValueError:

        print("Invalid version test pass")


def test_invalid_header_length():

    pkt = {

        'header': [2, 2],
```

```
        'entry': [[54321, 5]]
    }
    try:
        s.check_pkt(pkt)
        print("Invalid header length test , No error raised!!")
    except ValueError:
        print("Invalid header length test pass")

def test_empty_entry():
    pkt = {
        'header': [2, 2, 12345],
        'entry': []
    }
    try:
        s.check_pkt(pkt)
    except ValueError:
        print("Empty entry test pass")

def test_invalid_destination_id():
    pkt = {
        'header': [2, 2, 12345],
        'entry': [[-1, 5]]
    }
    try:
        s.check_pkt(pkt)
        print("Invalid destination ID test , No error raised!!")
    except ValueError:
        print("Invalid destination ID test pass")

def test_invalid_metric():
    pkt = {
        'header': [2, 2, 12345],
        'entry': [[54321, 17]]
    }
```

```
    try:
        s.check_pkt(pkt)
        print("Invalid metric test, No error raised!!")
    except ValueError:
        print("Invalid metric test pass")
```

```
test_set_poisoned_reverse pass.
jadeshi@jades-mbp assignment1 % /opt/homebrew/bin/python3 /Users/jadeshi/Desktop/COSC364/assignment1/test/test_rip_consistency.py
Invalid command test pass
Invalid version test pass
Invalid header length test pass
Empty entry test pass
Invalid destination ID test pass
Invalid metric test pass
```

Routing Table

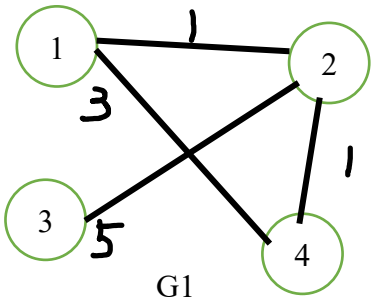
Test all the function in routing table. Ensure that each function behaves as expected when managing the routing table, including adding, removing, flagging, and updating routes.

```
def test_new_route():
    routing_table = {}
    destination = 3
    next_hop = 5
    cost = 2
    new_route(destination, next_hop, cost, routing_table)
    assert destination in routing_table
    assert routing_table[destination]['next_hop'] == next_hop
    assert routing_table[destination]['cost'] == cost
    assert not routing_table[destination]['garbage']
    print("test_new_route pass.")
def test_remove_route():
    routing_table = {3: {'next_hop': 5, 'cost': 2, 'garbage': False}}
    destination = 3
    remove_route(destination, routing_table)
    assert destination not in routing_table
    print("test_remove_route pass.")
def test_flag_garbage():
    routing_table = {3: {'next_hop': 5, 'cost': 2, 'garbage': False}}
    destination = 3
    flag_garbage(destination, routing_table)
    assert routing_table[destination]['garbage']
    print("test_flag_garbage pass.")
```

```
def test_set_infinity():
    routing_table = {3: {'next_hop': 5, 'cost': 2, 'garbage': False}}
    destination = 3
    set_infinity(destination, routing_table)
    assert routing_table[destination]['cost'] == METRIC_INFINITY
    print("test_set_infinity pass.")
```

Routing Algorithm

We build up an easy model G1 to test the basic functional for routing algorithm, for add new route and update when a shorter route been founded.



G1

First test with router 1 received the packet from 2, We expected the new route from 1 to 3 added.

```
table1 = {
    2: {'next_hop': 2, 'cost': 1, 'garbage': False, 'last_update_time': time.time()},
    4: {'next_hop': 2, 'cost': 1, 'garbage': False, 'last_update_time': time.time()}}
packet2 = {'header': [2, 2, 2], 'entry': [[1, 1],[3, 5],[4, 1]]}
def test_add_new_route():
    router_ID = 1
    new_routing_table, update = routing_algorithms(router_ID, table1, packet2)
    assert update == True
    assert new_routing_table[3]['cost'] == 6
    assert new_routing_table[3]['next_hop'] == 2
    assert new_routing_table[3]['garbage'] == False
    print("test_add_new_route passed.")
```

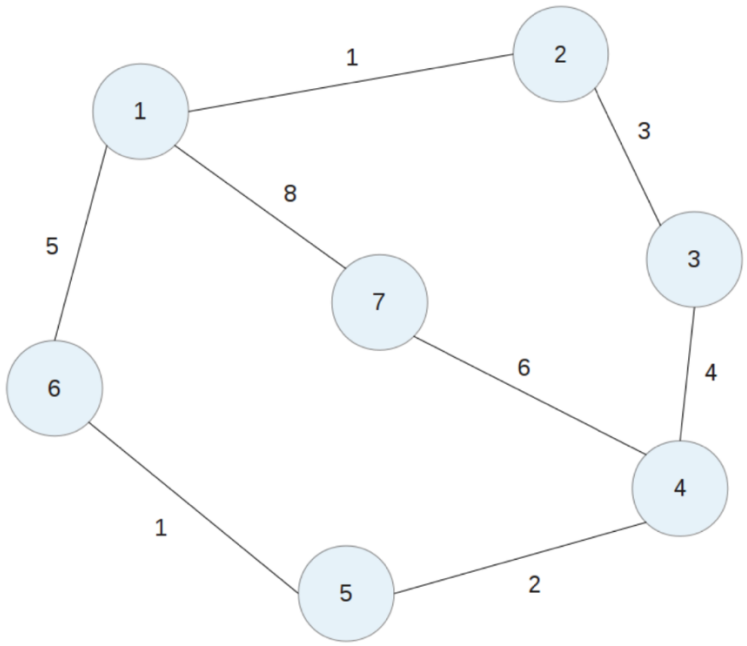
Also we test the Scenario for a shorter path been found, in this case the path from router1 to router 4 should been update through router 2 and give a shorter cost 2.

```
table11 = {
    2: {'next_hop': 2, 'cost': 1, 'garbage': False, 'last_update_time': time.time()},
```

```
    4: {'next_hop': 4, 'cost': 3, 'garbage': False, 'last_update_time': time.time()},
    3: {'next_hop': 2, 'cost': 6, 'garbage': False, 'last_update_time': time.time()}}
packet22 = {'header': [2, 2, 2], 'entry': [[1, 1],[3, 5],[4, 1]]}
def test_update_existing_route():
    router_ID = 1
    new_routing_table, update = routing_algorithms(router_ID, table11, packet22)
    assert update == True
    assert new_routing_table[4]['cost'] == 2, "Expected cost to be 6"
    assert new_routing_table[4]['next_hop'] == 2, "Expected next_hop to be 6"
    assert new_routing_table[4]['garbage'] == False, "Expected garbage to be False"
    print("test_update_existing_route passed.")
```

2.2 Integration Test

To ensure that components work together as expected and to test the server's functionality, including timer and socket interface. We utilize the example network provided in the assignment handout for our integration tests.



Example PARTLY TEST:

Run router 1:

Router 1 will start with an empty Initial Routing Table and will send RIP packets every 30 seconds to the ports specified in the configuration file: 2221, 6661, and 7771. Since no other routers are active, the routing table will remain empty until it receives further RIP packets.

```
jadeshi@jades-mbp assignment1 % python3 main.py router1.txt
 --------------Initial Routing Table--------------

####################Routing Table####################
#####################################################

!!!!!!!! 30s trigger update send to neighbors [2221, 6661, 7771]!!!!!!!!
```

Run router 2:

Router 2 should start with an empty Initial Routing Table and send RIP packet every 30 seconds to the port it read from configuration file: 1111, 3331.

```
jadeshi@jades-mbp assignment1 % python3 main.py router2.txt
 --------------Initial Routing Table--------------

####################Routing Table####################
#####################################################
```

With in next 30 seconds, Router 2 will receive a RIP packet from port 1111 (Router 1). It will process and decode this packet, then run the routing algorithm to update its own routing table.

```
-----------------------------------------------------
Received data from port 1111:
Header:
  Version: 1
  Type: 2
  Source_Router: 1
Entries:
  Destination: 2, Cost: 1
-----------------------------------------------------

@@@@@@@@@ Neighbor 1 is back to alive @@@@@@@@@
@@@@@@@@@@@@@@@@ Routing table update @@@@@@@@@@@@@@@@

####################Routing Table####################
               Destination: 1
   Next Hop: 1    Cost: 1    Garbage: False
   Last Update Time: 1744681630.8312
-----------------------------------------------------
#####################################################
```

Same as router 2, router 1 will receive a RIP packet from port 2221(router2) and will process and decode this packet to update the routing table.

The top 3 “ trigger update send to neighbours” are before router 2 opened, it keep periodically sent to each output port read from configuration file.

```
!!!!!!!! 30s trigger update send to neighbors [2221, 6661, 7771]!!!!!!!!

####################Routing Table####################
#####################################################

!!!!!!!! 30s trigger update send to neighbors [2221, 6661, 7771]!!!!!!!!

####################Routing Table####################
#####################################################

!!!!!!!! 30s trigger update send to neighbors [2221, 6661, 7771]!!!!!!!!

####################Routing Table####################
#####################################################


-----------------------------------------------------
Received data from port 2221:
Header:
  Version: 1
  Type: 2
  Source_Router: 2
Entries:
  Destination: 1, Cost: 1
-----------------------------------------------------

@@@@@@@@@@ Neighbor 2 is back to alive @@@@@@@@@@
@@@@@@@@@@@@@@@@@@ Routing table update @@@@@@@@@@@@@@@@@@

####################Routing Table####################
               Destination: 2
   Next Hop: 2    Cost: 1    Garbage: False
   Last Update Time: 1744681623.802022
-----------------------------------------------------
#####################################################
```

In each router's routing table update time should be the last time when received the RIP packet and the Garbage should be always False since they both alive.

Run router 3:
When Router 3 is activated, it sends a RIP packet to Router 2. Router 2 processes this packet, updates its routing table with the new path to Router 3, and includes this information in its next periodic update (30 seconds later) to Router 1. Consequently, Router 1 updates its table to include the path to Router 3.

This is the data we read from router 1 processed RIP packet and update it self's routing table with 3.

```
------------------------------------------------
Received data from port 2221:
Header:
  Version: 1
  Type: 2
  Source_Router: 2
Entries:
  Destination: 1, Cost: 1
  Destination: 3, Cost: 3
------------------------------------------------

!!!!!!!! 30s trigger update send to neighbors [2221, 6661, 7771]!!!!!!!!

####################Routing Table####################
               Destination: 2
   Next Hop: 2    Cost: 1    Garbage: False
   Last Update Time: 1744682126.819836
------------------------------------------------
               Destination: 3
   Next Hop: 2    Cost: 4    Garbage: False
   Last Update Time: 1744682126.819836
------------------------------------------------
#################################################
```

In the next periodic update, Router 1 will send a poisoned packet to Router 2, as it learned the route to Router 3 from Router 2. Router 2 will receive a RIP packet from port 1111 with the cost to Router 3 set to 16. Simultaneously, Router 3 will learn the path to Router 1 and send a poisoned packet to Router 2.
In the data read from router2 below, router 2 receives packets from both Router 3 and Router 1, where the cost to reach destinations 1 and 3 is set to 16. This matches our expectations, confirming that Router 2 has been effectively poisoned.

```
------------------------------------------------
Received data from port 3331:
Header:
  Version: 1
  Type: 2
  Source_Router: 3
Entries:
  Destination: 2, Cost: 3
  Destination: 1, Cost: 16
------------------------------------------------


------------------------------------------------
Received data from port 1111:
Header:
  Version: 1
  Type: 2
  Source_Router: 1
Entries:
  Destination: 2, Cost: 1
  Destination: 3, Cost: 16
------------------------------------------------
```

Shut down router 2 , to see how the reaction router 1 and router 3 will drop this route.
For **router 3**:
Since there was no update RIP received from router 2 so both route to router 2 and 1 ‘s update time will frozen until:
After 120s router 2 been shut , router 3 will set the cost to router 2 and router 1 to 16.

```
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Route to 2 expired 120s -> garbage collection
@@@@ router invalid trigger update send to neighbors@@@@

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Route to 1 expired 120s -> garbage collection
@@@@ router invalid trigger update send to neighbors@@@@
!!!!!!!! 30s trigger update send to neighbors [2222, 4441]!!!!!!!!

####################Routing Table####################
              Destination: 2
   Next Hop: 2    Cost: 16    Garbage: True
   Last Update Time: 1744682379.7754471
-------------------------------------------------
              Destination: 1
   Next Hop: 2    Cost: 16    Garbage: True
   Last Update Time: 1744682379.7754471
-------------------------------------------------
##################################################
```

After 180s router 2 been shut , router 3 will remove router 2 and router 1 from it routing table. So it ended with an empty routing table, because it need go through router 2 then can reach router 1.

```
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Route to 2 expired 180s -> remove_route
@@@@ router invalid trigger update send to neighbors@@@@

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Route to 1 expired 180s -> remove_route
@@@@ router invalid trigger update send to neighbors@@@@
!!!!!!!! 30s trigger update send to neighbors [2222, 4441]!!!!!!!!

####################Routing Table####################
##################################################
```

This will also happened to router 1.

```
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Route to 2 expired 120s -> garbage collection
@@@@ router invalid trigger update send to neighbors@@@@

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Route to 3 expired 120s -> garbage collection
@@@@ router invalid trigger update send to neighbors@@@@
!!!!!!!! 30s trigger update send to neighbors [2221, 6661, 7771]!!!!!!!!

####################Routing Table####################
               Destination: 2
   Next Hop: 2    Cost: 16    Garbage: True
   Last Update Time: 1744696927.918305
-------------------------------------------------
               Destination: 3
   Next Hop: 2    Cost: 16    Garbage: True
   Last Update Time: 1744696927.918305
-------------------------------------------------
##################################################


!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Route to 2 expired 180s -> remove_route
@@@@ router invalid trigger update send to neighbors@@@@

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Route to 3 expired 180s -> remove_route
@@@@ router invalid trigger update send to neighbors@@@@
!!!!!!!! 30s trigger update send to neighbors [2221, 6661, 7771]!!!!!!!!

####################Routing Table####################
##################################################
```

Run router 2 back:
Router1 will find the route from origin_routing_info and add the router 2 back, later router 2 find the way to router 3 so router 1 also add router 3 back.

```
@@@@@@@@@@ Neighbor 2 is back to alive @@@@@@@@@@
@@@@@@@@@@@@@@@@ Routing table update @@@@@@@@@@@@@@@@

####################Routing Table####################
               Destination: 2
   Next Hop: 2    Cost: 1    Garbage: False
   Last Update Time: 1744697251.1180549
-------------------------------------------------
#################################################

!!!!!!!! 30s trigger update send to neighbors [2221, 6661, 7771]!!!!!!!!

####################Routing Table####################
               Destination: 2
   Next Hop: 2    Cost: 1    Garbage: False
   Last Update Time: 1744697251.1180549
-------------------------------------------------
#################################################


-------------------------------------------------
Received data from port 2221:
Header:
  Version: 1
  Type: 2
  Source_Router: 2
Entries:
  Destination: 3, Cost: 3
  Destination: 1, Cost: 1
-------------------------------------------------

New route to 3 via 2 with cost 4 added.
Routing table update from 2221:

####################Routing Table####################
               Destination: 2
   Next Hop: 2    Cost: 1    Garbage: False
   Last Update Time: 1744697277.217655
-------------------------------------------------
               Destination: 3
   Next Hop: 2    Cost: 4    Garbage: False
   Last Update Time: 1744697277.217742
-------------------------------------------------
#################################################
```

Example OVERALL TEST:
We run all 7 router in sequence 1-7 and take some time to make the routing table converge.
All the router have 100% correctness for the final converging and give a solid result.
We take 2 router for example
The final routing table in router 1 :

```
####################Routing Table####################
               Destination: 2
   Next Hop: 2    Cost: 1    Garbage: False
   Last Update Time: 1744697622.797058
-------------------------------------------------
               Destination: 6
   Next Hop: 6    Cost: 5    Garbage: False
   Last Update Time: 1744697636.856758
-------------------------------------------------
               Destination: 7
   Next Hop: 7    Cost: 8    Garbage: False
   Last Update Time: 1744697642.87879
-------------------------------------------------
               Destination: 3
   Next Hop: 2    Cost: 4    Garbage: False
   Last Update Time: 1744697622.7971
-------------------------------------------------
               Destination: 5
   Next Hop: 6    Cost: 6    Garbage: False
   Last Update Time: 1744697636.8568032
-------------------------------------------------
               Destination: 4
   Next Hop: 6    Cost: 8    Garbage: False
   Last Update Time: 1744697636.856819
-------------------------------------------------
#################################################
```

The final routing table in router 4 :

```
####################Routing Table####################
               Destination: 5
   Next Hop: 5    Cost: 2    Garbage: False
   Last Update Time: 1744697668.9938529
-------------------------------------------------
               Destination: 7
   Next Hop: 7    Cost: 6    Garbage: False
   Last Update Time: 1744697669.99574
-------------------------------------------------
               Destination: 3
   Next Hop: 3    Cost: 4    Garbage: False
   Last Update Time: 1744697689.0628061
-------------------------------------------------
               Destination: 2
   Next Hop: 3    Cost: 7    Garbage: False
   Last Update Time: 1744697689.0628061
-------------------------------------------------
               Destination: 1
   Next Hop: 3    Cost: 8    Garbage: False
   Last Update Time: 1744697689.0628061
-------------------------------------------------
               Destination: 6
   Next Hop: 5    Cost: 3    Garbage: False
   Last Update Time: 1744697668.9938529
-------------------------------------------------
#################################################
```

Next we turn off router 2 , wait for 180s until this router been fully removal from the routing table in neighbours.

For **router 1** since it cut the original way to router 3.It will fin a new route router 1 - 6 – 5 – 4 – 3, with cost 5+1+2+4 = 12. Also, the reachability to router 2 been removal.

```
####################Routing Table####################
               Destination: 6
   Next Hop: 6    Cost: 5    Garbage: False
   Last Update Time: 1744698611.379456
-------------------------------------------------
               Destination: 7
   Next Hop: 7    Cost: 8    Garbage: False
   Last Update Time: 1744698602.3486662
-------------------------------------------------
               Destination: 3
   Next Hop: 6    Cost: 12    Garbage: False
   Last Update Time: 1744698611.379456
-------------------------------------------------
               Destination: 5
   Next Hop: 6    Cost: 6    Garbage: False
   Last Update Time: 1744698611.379456
-------------------------------------------------
               Destination: 4
   Next Hop: 6    Cost: 8    Garbage: False
   Last Update Time: 1744698611.379456
-------------------------------------------------
################################################
```

For **router 4 ,** it use router 3 as a out gate then trivial through router 2 to router 1. From router 2 been cut ,it need to learn a new path to touter 1. It will find a better route with next hop router 5 ,and cost 2+1+5 =8. Plus, the reachability to router 2 been removal.

```
####################Routing Table####################
               Destination: 5
   Next Hop: 5    Cost: 2    Garbage: False
   Last Update Time: 1744698638.485485
-------------------------------------------------
               Destination: 7
   Next Hop: 7    Cost: 6    Garbage: False
   Last Update Time: 1744698634.470955
-------------------------------------------------
               Destination: 3
   Next Hop: 3    Cost: 4    Garbage: False
   Last Update Time: 1744698608.3724039
-------------------------------------------------
               Destination: 2
   Next Hop: 5    Cost: 9    Garbage: False
   Last Update Time: 1744698638.4855318
-------------------------------------------------
               Destination: 1
   Next Hop: 5    Cost: 8    Garbage: False
   Last Update Time: 1744698638.485535
-------------------------------------------------
               Destination: 6
   Next Hop: 5    Cost: 3    Garbage: False
   Last Update Time: 1744698638.485485
-------------------------------------------------
################################################
```

We conducted comprehensive tests by adding and removing each router to evaluate partial and complete routing relationships. This approach provided valuable insights for debugging, and ultimately, we achieved a 100% pass rate in the final tests.

3 Configuration File

We use .txt syntax for the configuration file which achieved humans read and edited. Followed is a sample of configuration file for router 1.

---

router-id 1
input-ports 1111, 1112, 1113
output-ports 2221-1-2, 6661-5-6, 7771-8-7

---

4 Source Code

src/config_processer.py

```
import re

import os


def read_config(filename):

    #dictionary to store the router_id and input_ports

    # read some file of router_id and input_ports

    router_txt=[]

    router_id = 0

    input_ports=[]

    output_ports=[]

    file_path = os.path.join(os.path.dirname(__file__), '..', 'router', filename)

    file=open( file_path,'r')

    for line in file.readlines():

        line = re.split(', | |\n',line)

        router_txt.append(line)

    #print(f"Config file readed: {router_txt}")

    if router_txt == []:

        raise ValueError("ERROR: Invalid router config file, empty file")

    if len(router_txt) != 3:

        raise ValueError("ERROR: Invalid router config file, must be 3 lines")
```

```
    #get router_id
    router_id=check_router_id(router_txt[0])
    #print(f"Router_id : {router_id}")



    #get input_ports
    input_ports=check_input_ports(router_txt[1])
    #print(f"Input_ports : {input_ports}")

    #get output_ports
    output_ports=check_output_ports(router_txt[2],input_ports)
    #print(f"Output_ports = [peer_port, metric, peer_ID]: {output_ports}")

    config= {
    'router_id': router_id,
    'input_ports': input_ports,
    'output_ports': output_ports
    }
    return config

def check_router_id(line):
        if len(line) != 3:
            raise ValueError("ERROR: Invalid router_id format, must be 'router_id
<router_id>'")
        if line[0] != 'router-id':
            raise ValueError(f"ERROR: Invalid router_id format, must be 'router_id
<router_id>', not {line[0]}")
        if line[1].isalpha():
            raise ValueError(f"ERROR: Invalid router_id format, router_id must be not an
alphabetic string, not {line[1]}")
        if line[1].isdigit():
            if int(line[1])<=0:
                raise ValueError(f"ERROR: Invalid router_id format, router_id must be more
than zero, not {line[1]}")
```

```
            if int(line[1])>64000:
                raise ValueError(f"ERROR: Invalid router_id format, router_id must be less
than 64000, not {line[1]}")
            line[1] = int(line[1])
        else:
            if line[1].startswith('-'):
                raise ValueError(f"ERROR: Invalid router_id format, router_id must be
positive integer, not {line[1]}")
            raise ValueError(f"ERROR: Invalid router_id format, router_id must be an integer,
not {line[1]}")
        return line[1]
def check_input_ports(line):
        input_list = []
        if len(line) <= 2:
            raise ValueError("ERROR: Invalid input_ports format, At least one input port is
required")
        if line[0] != 'input-ports':
            raise ValueError(f"ERROR: Invalid input_ports format, the first one is 'input-
ports', not {line[0]}")
        for port in line[1:-1]:
            if port.isalpha():
                raise ValueError(f"ERROR: Invalid input_ports format, input_ports must be not
an alphabetic string, not {port}")
            if port.isdigit():
                if int(port)<1024:
                    raise ValueError(f"ERROR: Invalid input_ports format, input_ports must be
more then 1024, not {port}")
                if int(port)>64000:
                    raise ValueError(f"ERROR: Invalid input_ports format, input_ports must be
less than 64000, not {port}")
                input_list.append(int(port))
            else:
                if port.startswith('-'):
```

```
                    raise ValueError(f"ERROR: Invalid input_ports format, input_ports must be
more than 1024 and not negative, not {port}")
                raise ValueError(f"ERROR: Invalid input_ports format, input_ports must be an
integer, not {port}")
        if len(input_list) !=len(set(input_list)):
            raise ValueError(f"ERROR: Invalid input_ports format, input_ports must be
unique")
        return input_list

def check_output_ports(line,input_ports):
        output_list = []
        check_same_ports = []
        if len(line) <= 2:
            raise ValueError("ERROR: Invalid output_ports format, At least one output port is
required")
        if line[0] != 'output-ports':
            raise ValueError(f"ERROR: Invalid output_ports format, the first one is 'output-
ports', not {line[0]}")

        for port in line[1:-1]:
            port = re.split('-',port)
            if len(port) != 3:
                raise ValueError(f"ERROR: Invalid output-port format, must be 'output-ports
<peer_port> <metric> <peer_ID>'")
            if port[0].isalpha() or port[1].isalpha() or port[2].isalpha():
                raise ValueError(f"ERROR: Invalid output_ports format, output_ports must be
not an alphabetic string, not {port}")

            # check port[1]
            if port[1].isdigit():
                if int(port[1])<0:
                    raise ValueError(f"ERROR: Invalid output_ports format, metric must be
more then 0, not {port[1]}")
                if int(port[1])>16:
```

```
                    raise ValueError(f"ERROR: Invalid output_ports format, metric must be
less than 16, not {port[1]}")
                port[1] = int(port[1])
            # check port[2]
            if port[2].isdigit():
                if int(port[2])<=0:
                    raise ValueError(f"ERROR: Invalid output_ports format, router_id must be
more than zero, not {port[2]}")
                if int(port[2])>64000:
                    raise ValueError(f"ERROR: Invalid output_ports format, router_id must be
less than 64000, not {port[2]}")
                port[2] = int(port[2])
            else:
                if line[1].startswith('-'):
                    raise ValueError(f"ERROR: Invalid output_ports format, router_id must be
positive integer, not {port[2]}")
                raise ValueError(f"ERROR: Invalid output_ports format, router_id must be an
integer, not {port[2]}")
            # check port[0]
            if port[0].isdigit():
                if int(port[0])<1024:
                    raise ValueError(f"ERROR: Invalid output_ports format, port must be more
then 1024, not {port[0]}")
                if int(port[0])>64000:
                    raise ValueError(f"ERROR: Invalid output_ports format, port must be less
than 64000, not {port[0]}")
                if int(port[0]) in input_ports:
                    raise ValueError(f"ERROR: Invalid output_ports format, port must not be
the same as input_ports, not {port[0]}")
                for i in check_same_ports:
                    if int(port[0]) == int(i):
                        raise ValueError(f"ERROR: Invalid output_ports format, port must be
unique, {port[0]} is already used")
```

```
                check_same_ports.append(port[0])

            else:
                if port[0].startswith('-'):
                    raise ValueError(f"ERROR: Invalid output_ports format, port must be
positive integer, not {port[0]}")
                raise ValueError(f"ERROR: Invalid output_ports format, port must be an
integer, not {port[0]}")
            # make output_list

            output_list.append([int(port[0]),int(port[1]),int(port[2])])


        return output_list
```

src/routing_table.py

```
"""
    Routing Table Structure
    +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
    |   Dest(key) |  Next Hop | Cost | update timer  |  garbage   |
    +-------------+-----------+------+---------------+------------+
"""
import time

METRIC_INFINITY = 16

def new_route(destination, next_hop, cost,routing_table,time = time.time(), garbage=False):
    """Add or update a route in the routing table."""
    if not (1 <= cost < METRIC_INFINITY):
        raise ValueError(f"ERROR: Invalid metric {cost}. Must be in range 1-15.")
    routing_table[destination] = {
    'next_hop': next_hop,
```

```
    'cost': cost,
    'last_update_time': time,
    'garbage': garbage,
    }

def remove_route(destination,routing_table):
    """Remove a route from the routing table."""
    if destination in routing_table:
        del routing_table[destination]
        print("  ")
        print("!"*40)
        print(f"Route to {destination} expired 180s -> remove_route")
    return routing_table

def flag_garbage(destination,routing_table):
    """ Flag a route as garbage. """
    if destination in routing_table:
        routing_table[destination]['garbage'] = True
        print("  ")
        print("!"*40)
        print(f"Route to {destination} expired 120s -> garbage collection")
    return routing_table

def set_infinity(destination,routing_table):
    """ Set a route's metric to infinity. """
    if destination in routing_table:
        routing_table[destination]['cost'] = METRIC_INFINITY
    return routing_table
```

src/RIP_packet.py

```
"""
    Send to each neiboure for update the
```

```
    RIP Packet Structure
    +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
    | command : 1/2   | version : 2  |      must be zero (2)       |
    +---------------+---------------+-------------------------------+
    |                  RIP Entry (20 bytes)                         |
    |                [(destination , metric),()...]                  |
    +---------------------------------------------------------------+
    command = 1 for request, 2 for response
    version = 2
"""
# Only use COMMAND_RESPONSE

COMMAND_RESPONSE = 2
VERSION = 2
METRIC_INFINITY = 16

def RIP_header(router_id):
    """Build up rip packet header and entry"""
    # rip packet header = [command, version, source]
    source = router_id
    command = COMMAND_RESPONSE
    header = [command, VERSION, source]
    return header

def RIP_entry(table):
    # table = (destination:{ next_hop, cost, garbage flag})
    entry = []
    for destination, info in table.items():
        if info['garbage'] != True:

            metric = info['cost']
            entry.append([destination, metric])
    return entry
```

```
def rip_packet(router_id, table):
    """Full RIP packet."""
    header = RIP_header(router_id)
    entries = RIP_entry(table)
    packet = {'header':header, 'entry':entries}
    return packet


def set_poisoned_reverse(router_ID, table, port_id):
    """Set poisoned reverse for routes learned from a specific neighbor."""
    poisoned_entries = []
    for destination, route_info in table.items():
        if (route_info['next_hop'] == port_id) and destination != port_id:
            # Set the metric to infinity for this route
            poisoned_entries.append((destination, 16))
        else:
            # Keep the original metric for other routes
            poisoned_entries.append((destination, route_info['cost']))
    # Construct the packet with the poisoned reverse entries
    poisoned_packet = {
        'header': RIP_header(router_ID),
        'entry': poisoned_entries
    }
    return poisoned_packet
```

src/routing_algorithm.py

```
import time
import routing_table as rt
METRIC_INFINITY = 16

def routing_algorithms(router_ID, table, packet):
    """Return a format of updated routing table."""
```

```
    update = False
    # Received a packet, update the routing table
    # Record where it came from -> src_router_id
    src_router_id = packet['header'][2]
    # Record 'entry', e.g., [(2, 1), (6, 5), (7, 8)]

    for entry in packet['entry']:
        destination = entry[0]
        metric = entry[1]
        # If the destination is not the current router_ID, avoid using itself
        if destination != router_ID:
            # If metric is greater than 16, metric = inf = 16
            new_cost = metric + table.get(src_router_id, {}).get('cost', 0)
            if new_cost > METRIC_INFINITY:
                new_cost = METRIC_INFINITY

            # Scenario 1: If the destination is not in the current routing table, add a new
destination
            if destination not in table:
                if new_cost < METRIC_INFINITY:
                    print(f"New route to {destination} via {src_router_id} with cost
{new_cost} added.")
                    rt.new_route(destination, src_router_id, new_cost,table,time =
time.time(), garbage=False)
                    #4.2 Implement triggered updates only when routes become invalid
                    update = True

            else:
                # Scenario 2: If the next_hop is the same and there is a better path, update
                if src_router_id == table[destination]['next_hop']:
                    if new_cost < table[destination]['cost']:
                        table[destination]['cost'] = new_cost
                        table[destination]['garbage'] = False
                        table[destination]['last_update_time'] = time.time()
```

```
                        update = True
                # Scenario 3: If the next_hop is different and there is a better path, update
to use src_router_id as next_hop
                if new_cost < table[destination]['cost']:
                    table[destination]['next_hop'] = src_router_id
                    table[destination]['cost'] = new_cost
                    table[destination]['garbage'] = False
                    table[destination]['last_update_time'] = time.time()
                    update = True

    return table, update


def timer_update(table, port_id,pkt):
    current_time = time.time()
    pkt_destinations =[]
    src_router_id = pkt['header'][2]
    for entry in pkt['entry']:
        destination = entry[0]
        pkt_destinations.append(destination)
    for destination, route_info in table.items():
        if route_info['next_hop'] == port_id:
            if destination in pkt_destinations or destination == src_router_id:
                table[destination]['last_update_time'] = current_time
    return table
```

src/Server.py

```
"""
This acting as server in a router.
Server creates as many UDP sockets as it has input ports and binds
```

```
one socket to each input port. One of the input sockets can be used for
sending UDP datagrams to neighbors.
"""
import socket
import select
import time
import random


import config_processer as cfg
import routing_algorithm as ra
import routing_table as table
import RIP_packet as packet

global router_ID
global neighbor_mapping


router_ID = None
#neighbor_mapping saves the mapping between port and router ID
#neighbor_mapping = {port: neighbor_id}
neighbor_mapping={}

def init(config_filename):
    """Initialize the routing table inside a router from config file,
    also grnerate the first round RIP packet , this pkt will pass to send to the neighbors in
port
    """
    global router_ID
    global neighbor_mapping

    # Use table format to store a original routing information.
    origin_routing_info = {}

    config = cfg.read_config(config_filename)
```

```
    router_ID = config['router_id']
    input_ports = config['input_ports']
    neighbor_port = [i[0] for i in config['output_ports']]

    for router in config['output_ports']:
        #Output_ports = [peer_port, cost, peer_ID]
        peer_port = router[0]
        cost = router[1]
        peer_ID = router[2]
        neighbor_mapping[peer_port] = peer_ID
        table.new_route(peer_ID, peer_ID, cost,origin_routing_info)

    #rip_pkt = packet.rip_packet(router_ID, origin_routing_info[neighbor_mapping[peer_port]])
    return origin_routing_info,input_ports, neighbor_port

def pkt_to_str(rip_pkt):
    """Convert a RIP packet from dictionary to a string for sending over UDP."""
    header_str = ','.join(map(str, rip_pkt['header']))
    entries_str = ';'.join(','.join(map(str, entry)) for entry in rip_pkt['entry'])
    packet_str = f'header={header_str} || entries={entries_str}'
    return packet_str

def str_to_pkt(pkt_str):
    """Convert a RIP packet from a string to a dictionary for processing."""
    parts = pkt_str.split(" || ")
    header_str = parts[0].split("=")[1]
    entries_str = parts[1].split("=")[1]
    header = list(map(int, header_str.split(",")))
    entries = [list(map(int, entry.split(","))) for entry in entries_str.split(";") if entry]
    return {'header': header, 'entry': entries}


def create_and_bind(input_ports):
    """Create and bind UDP sockets for each input port."""
```

```
    sockets = []
    for port in input_ports:
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.bind(('127.0.0.1', port))
        sockets.append(sock)
    return sockets

def send_to_neighbors(sock, neighbor_port, rip_pkt):
    """Send the rip pk to each neighbor using the specified socket."""
    neighbor_ip = '127.0.0.1'
    str_pck = pkt_to_str(rip_pkt)
    sock.sendto(str_pck.encode(), (neighbor_ip, neighbor_port))

def trigger_update(routing_table,send_socket,neighbors):
    """ triverse each neighbors, Get rip pk ready then to send to neighbors port."""
    global router_ID
    global neighbor_mapping
    for neighbor_port in neighbors:
        neighbor_id = neighbor_mapping[neighbor_port]
        rip_pkt = packet.set_poisoned_reverse(router_ID, routing_table, neighbor_id)
        send_to_neighbors(send_socket, neighbor_port, rip_pkt)

def add_neighbor_router_back(routing_table,neighbor_id,origin_routing_info,pkt):
    """
    If a neighbor router is back to alive
    Add this neighbor router back to the routing table.
    The rest of the routing table will be updated by the routing algorithm.
    """
    current_time = time.time()
    update = False
    #scenario 1: Back connection with this router before removing it
    if neighbor_id in routing_table:
        # 1. Set the garbage flag to False, and update the last update time
        if routing_table[neighbor_id]['garbage'] == True:
```

```
            routing_table[neighbor_id]['garbage'] = False
            routing_table[neighbor_id]['last_update_time'] = current_time
            routing_table[neighbor_id]['cost'] = origin_routing_info[neighbor_id]['cost']
            update = True

        # 2. If the original cost of this neighbor router is smaller than the cost it in the
routing table
        #    Use the original cost
        if routing_table[neighbor_id]['cost'] > origin_routing_info[neighbor_id]['cost']:
            routing_table[neighbor_id]['cost'] = origin_routing_info[neighbor_id]['cost']
            routing_table[neighbor_id]['next_hop'] = neighbor_id
            routing_table[neighbor_id]['last_update_time'] = current_time
            update = True

    #scenario 2:  if the neighbor router is been removal
    if neighbor_id  not in routing_table:
       routing_table[neighbor_id] = {'next_hop': neighbor_id, 'cost':
origin_routing_info[neighbor_id]['cost'], 'garbage': False, 'last_update_time': current_time}
       update = True
    return update

def check_alive(destinations_to_check,routing_table,send_socket,neighbors):
    """Check if the route in the routing table is still alive."""
    current_time = time.time()
    for destination in destinations_to_check:
        route_info = routing_table[destination]
        #180s not recive from this port:
        if current_time - route_info['last_update_time'] > 120 and route_info['garbage'] ==
False:
            routing_table = table.set_infinity(destination,routing_table)
            routing_table = table.flag_garbage(destination,routing_table)
            print(f"@@@@ router invalid trigger update send to neighbors@@@@")
            trigger_update(routing_table,send_socket,neighbors)
```

```
        if route_info['garbage'] == True and current_time - route_info['last_update_time'] >=
180:
            routing_table = table.remove_route(destination,routing_table)
            print(f"@@@@ router invalid trigger update send to neighbors@@@@")
            trigger_update(routing_table,send_socket,neighbors)
    return routing_table

def print_routing_table(routing_table):
    """Nicer look in print the contents of the routing table."""
    print("  ")
    print("#" * 20 + "Routing Table" + "#" * 20)
    for destination, info in routing_table.items():
        print(f"               Destination: {destination}")
        print(f"   Next Hop: {info['next_hop']}    Cost: {info['cost']}    Garbage:
{info['garbage']}")
        print(f"   Last Update Time: {info['last_update_time']}")
        print("-" * 50)
    print("#" * 50)
    print("  ")

def print_RIP(receive_port,pkt):
    """Nicer look in print the contents of a received RIP packet."""
    print("  ")
    print("-" * 50)
    print(f"Received data from port {receive_port}:")
    print("Header:")
    print(f"  Version: {pkt['header'][0]}")
    print(f"  Type: {pkt['header'][1]}")
    print(f"  Source_Router: {pkt['header'][2]}")
    print("Entries:")
    for entry in pkt['entry']:
        print(f"  Destination: {entry[0]}, Cost: {entry[1]}")
    print("-" * 50)
    print("  ")
```

```
def check_pkt(pkt):
    """Check if the received packet is valid."""
    if 'header' not in pkt or 'entry' not in pkt:
        raise ValueError("ERROR: Invalid packet format,must contain 'header' and 'entry'")
    #check header
    if len(pkt['header']) != 3:
        raise ValueError("ERROR: Invalid packet header length, must be 3")
    if pkt['header'][0] !=2:
        raise ValueError("ERROR: Invalid command, must be 2")
    if pkt['header'][1] != 2:
        raise ValueError("ERROR: Invalid version, must be 2")
    if pkt['header'][2] <= 0:
        raise ValueError("ERROR: Invalid source router ID, must be positive integer")
    if pkt['header'][2] > 64000:
        raise ValueError("ERROR: Invalid source router ID, must be less than 64000")
    #check entry
    if len(pkt['entry']) == 0:
        raise ValueError("ERROR: Invalid packet entry, must contain at least one entry")
    for entry in pkt['entry']:
        if len(entry) != 2:
            raise ValueError("ERROR: Invalid entry , must be [destination, metric] ")
        if entry[0] <= 0:
            raise ValueError("ERROR: Invalid destination router ID, must be positive
integer")
        if entry[0] > 64000:
            raise ValueError("ERROR: Invalid destination router ID, must be less than 64000")
        if entry[1] < 0 or entry[1] > 16:
            raise ValueError("ERROR: Invalid metric, must be in range 0-16")
    return pkt


def main(config_filename):
    """Run the server to listen on multiple UDP sockets and send to neighbors."""
    global router_ID
```

```
    global neighbor_mapping

######################-------------init------------#######################
    origin_routing_info,input_ports, neighbors = init(config_filename)
    # Creat a empty routing table
    routing_table = {}
    print(" --------------Initial Routing Table--------------")
    print_routing_table(routing_table)
    sockets = create_and_bind(input_ports)
    send_socket = sockets[0]  # First socket for sending
    for neighbor_port in neighbors:
        #Create single route RIP packet for each neighbor and send
        neighbor_id = neighbor_mapping[neighbor_port]
        neighbor_routing_info = {neighbor_id:origin_routing_info[neighbor_id]}
        rip_pkt = packet.rip_packet(router_ID, neighbor_routing_info)
        send_to_neighbors(send_socket, neighbor_port, rip_pkt)

######################-------------init------------#######################

    next_periodic_update_time = time.time() + 30 + random.uniform(-5, 5)
    next_sec = time.time() + 1

#------------------------------deamon loop-----------------------------------------------
--#
    try:
        while True:
            current_time = time.time()

            if  current_time >= next_periodic_update_time:
                    # Send the routing table to all neighbors every 30 seconds
                trigger_update(routing_table,send_socket,neighbors)
                print(f"!!!!!!!! 30s trigger update send to neighbors {neighbors}!!!!!!!!")
                print_routing_table(routing_table)
                next_periodic_update_time = current_time + 30 + random.uniform(-5, 5)
```

```
            if current_time >= next_sec:
                # avoid modifying the dictionary while iterating over it
                destinations_to_check = list(routing_table.keys())
                routing_table =
check_alive(destinations_to_check,routing_table,send_socket,neighbors)
                next_sec = current_time + 1

#------------------------------listen-----------------------------------------------------#
            #wait for any socket to have data
            readable, _writable_, _exceptional_ = select.select(sockets, [], [],1)
            for sock in readable:
                data, addr = sock.recvfrom(1024)  # Buffer size is 1024 bytes
                pkt = str_to_pkt(data.decode())
                receive_port = addr[1]

                try:
                    valid_pkt = check_pkt(pkt)
                except ValueError as error:
                    print(error)
                    continue

                print_RIP(receive_port,valid_pkt)

                if receive_port not in neighbor_mapping:
                    neighbor_mapping[receive_port] = valid_pkt['header'][2]

                neighbor_id = neighbor_mapping[receive_port]

                routing_table = ra.timer_update(routing_table,neighbor_id,valid_pkt)
                update =
add_neighbor_router_back(routing_table,neighbor_id,origin_routing_info,valid_pkt)
                if update:
                    print(f"@@@@@@@@@ Neighbor {neighbor_id} is back to alive @@@@@@@@@")
                    print(f"@@@@@@@@@@@@@@@@@ Routing table update @@@@@@@@@@@@@@@@@")
```

```
                    print_routing_table(routing_table)
                routing_table, update= ra.routing_algorithms(router_ID ,routing_table,
valid_pkt)
                if update:
                    print(f"Routing table update from {receive_port}:")
                    print_routing_table(routing_table)
 #-------------------------------------------------------------------------------#

    except KeyboardInterrupt:
        print("Server shutting down.")
    finally:
        for sock in sockets:
            sock.close()
```

src/main.py

```
import sys
import Server as s

#python3 main.py ../router/router1.txt
#python3 main.py ../router/router2.txt
#python3 main.py ../router/router3.txt
#python3 main.py ../router/router4.txt
#python3 main.py ../router/router5.txt
#python3 main.py ../router/router6.txt
#python3 main.py ../router/router7.txt

def main():
    if len(sys.argv) != 2:
        print("Usage: python script_name.py <config_filename>")
        sys.exit(1)

    config_filename = sys.argv[1]
    s.main(config_filename)
```

```python
if __name__ == '__main__':
    main()
```